

Experiment 30

You and your team are working on a project. One team member adds a new feature to the project. You need to fetch the latest changes, create a new branch, and implement your feature without affecting the existing code.

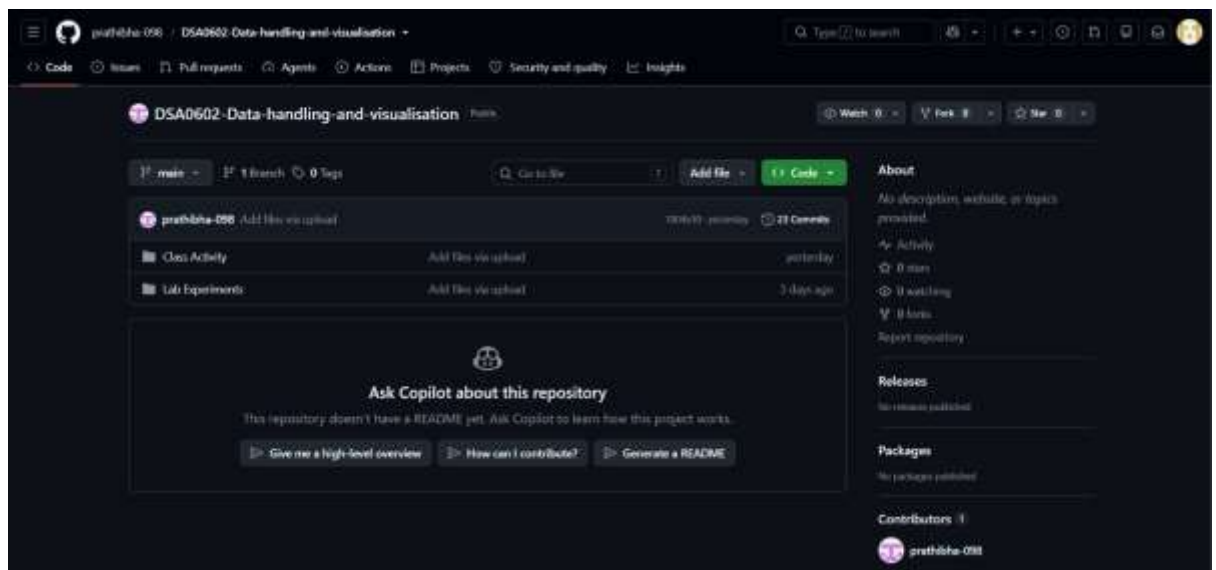
Tasks:

- Fork a repository (you can use an existing repository or a partner's repository).
- Clone the repository to your local machine.
- Create a new branch for your feature.
- Implement your changes and push them to your forked repository.
- Submit a pull request to the original repository.

Steps:

Step 1: Open the Original GitHub Repository

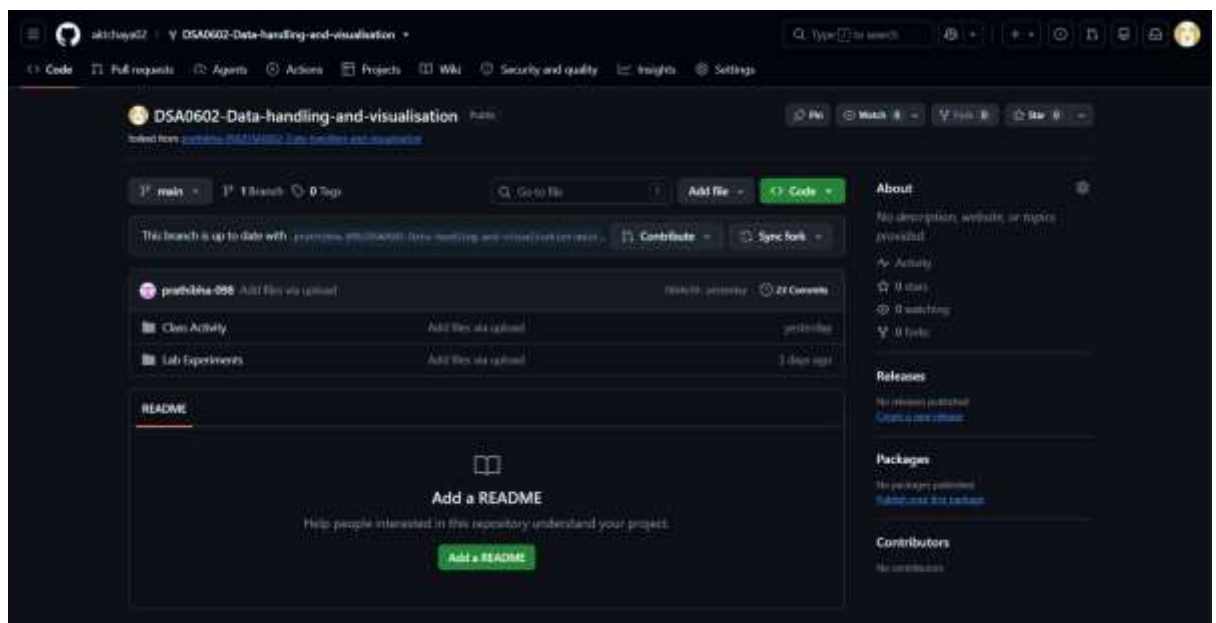
- Open the original repository provided by the team member/partner on GitHub.
- The repository contains the project files and the README.md file.



Step 2: Fork the Repository

- Open the original repository on GitHub.
- Click the **Fork** button.
- Select your GitHub account as the owner.
- Keep the repository name.
- Click **Create fork**.

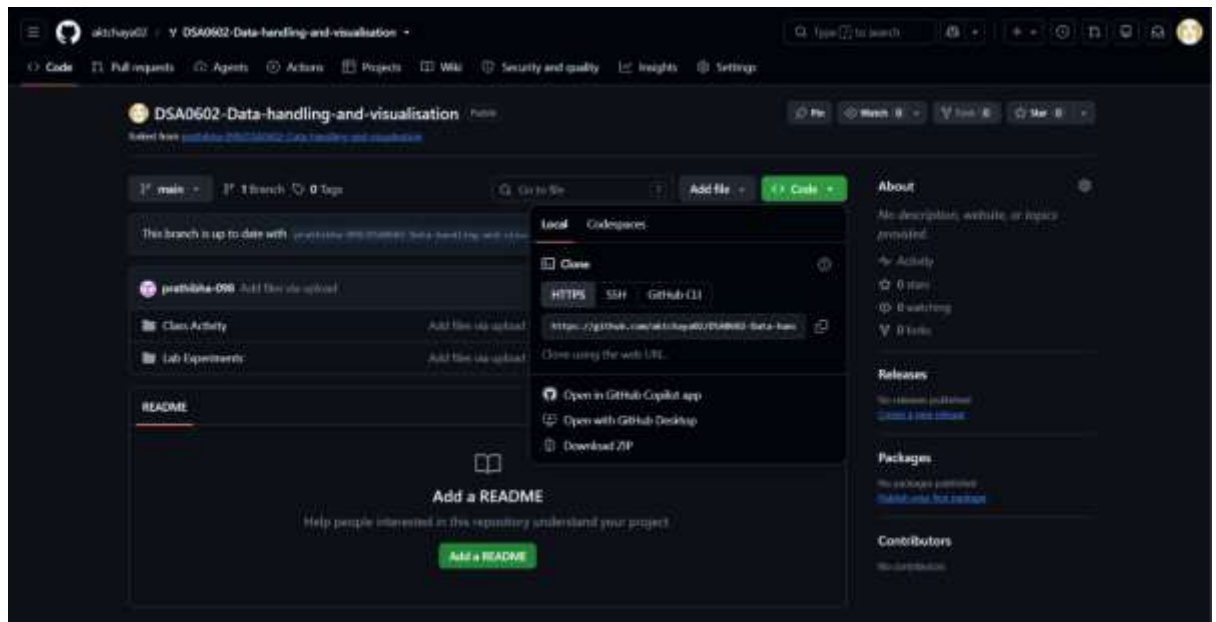
A copy of the original repository is created under your GitHub account.



Step 3: Copy the Repository URL

- Open the forked repository.
- Click the **Code** button.
- Select **HTTPS**.
- Click the copy button to copy the repository URL.

`https://github.com/aktchaya02/DSA0602-Data-handling-and-visualisation.git`



Step 4: Clone the Repository

- Open Visual Studio Code and create/open the terminal.
- Run:


```
git clone https://github.com/aktchaya02/DSA0602-Data-handling-and-visualisation.git
```
- Then enter the repository folder:


```
cd Github-Collaboration-Lab
```
- Check the repository status:


```
git status
```

```
PS C:\AKTCHAYA A> git clone https://github.com/aktchaya02/DSA0602-Data-handling-and-visualisation.git
Cloning into 'DSA0602-Data-handling-and-visualisation'...
remote: Enumerating objects: 67, done.
remote: Counting objects: 100% (67/67), done.
remote: Compressing objects: 100% (61/61), done.
remote: Total 67 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (67/67), 10.57 MiB | 217.00 KiB/s, done.
Resolving deltas: 100% (9/9), done.
```

```
PS C:\AKTCHAYA A> cd Github-Collaboration-Lab-1
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git status
On branch feature-login
Your branch is up to date with 'origin/feature-login'.
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    1NPDPRO5.JPG
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

Step 5: Configure the Original Repository as Upstream

- Check the remote repositories:
`git remote -v`
- Add the original repository as upstream:
`git remote add upstream https://github.com/PartnerUsername/Github-Collaboration-Lab.git`
- Replace PartnerUsername with the GitHub username of the repository owner.
- Check the remote repositories again:
`git remote -v`
- The output should contain:

origin

upstream

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git remote -v
origin https://github.com/aktchaya02/Github-Collaboration-Lab-1.git (fetch)
origin https://github.com/aktchaya02/Github-Collaboration-Lab-1.git (push)
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git remote add upstream https://github.com/prathibha-098/Github-Collaboration-Lab-1.git
usage: git remote add [<options>] <name> <url>

    -f, --[no-]fetch          fetch the remote branches
    --[no-]tags              import all tags and associated objects when fetching
                             or do not fetch any tag at all (--no-tags)
    -t, --[no-]track <branch> branch(es) to track
    -m, --[no-]master <branch> master branch
    --[no-]mirror[=(push|fetch)]
                             set up remote as a mirror to push to or fetch from

PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git remote -v
origin https://github.com/aktchaya02/Github-Collaboration-Lab-1.git (fetch)
origin https://github.com/aktchaya02/Github-Collaboration-Lab-1.git (push)
upstream https://github.com/prathibha-098/Github-Collaboration-Lab-1.git (fetch)
upstream https://github.com/prathibha-098/Github-Collaboration-Lab-1.git (push)
```

Step 6: Fetch the Latest Changes

- To obtain the latest changes from the original repository, execute:
`git fetch upstream`

This downloads the latest information from the original repository without directly changing the working files.

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git fetch upstream
fatal: 'upstream' does not appear to be a git repository
fatal: Could not read from remote repository.
```

Please make sure you have the correct access rights
and the repository exists.

Step 7: Update the Main Branch

- Switch to the main branch:
`git checkout main`
- Merge the latest changes from the original repository:
`git merge upstream/main`
- If there are no new changes, Git may display:
Already up to date.

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git merge upstream/main
merge: upstream/main - not something we can merge
```

Step 8: Create a New Feature Branch

- Create a separate branch for implementing the new feature.
`git checkout -b feature-update`
- Check the available branches:
`git branch`
- It show like this:
feature-update
main

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git checkout -b feature-collaboration
Switched to a new branch 'feature-collaboration'
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git branch
* feature-collaboration
  feature-login
  main
```

Step 9: Implement the Collaboration Feature

- Open the project in Visual Studio Code.
- The existing README.md file already contains information about the GitHub collaboration experiment.
- The new collaboration feature was added at the end of the existing README file.
- The following section was added:
Collaboration Feature

This section demonstrates the GitHub collaboration workflow.

- Created a separate feature branch.
- Implemented changes without affecting the main branch.
- Committed and pushed the changes to the forked repository.
- Submitted a pull request to the original repository.
- The existing README content was retained, and the new section was added without deleting the previous content.

```
1  # Github-Collaboration-Lab-1
12
13  ## Git and GitHub Experiment
14
15  This repository demonstrates modifying,
16  committing, and pushing changes using Git.
17
18  ## Collaboration Feature
19
20  This section demonstrates the GitHub collaboration workflow.
21
22  - Created a separate feature branch.
23  - Implemented changes without affecting the main branch.
24  - Committed and pushed the changes to the forked repository.
25  - Submitted a pull request to the original repository.
```

Step 10: Check the Modified Files

- After modifying README.md, check the repository status:
git status
- Git displays the modified files that are ready to be staged.

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git status
On branch feature-collaboration
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        INPDPRO5.JPG

no changes added to commit (use "git add" and/or "git commit -a")
```

Step 11: Stage the Changes

- Add the modified files to the staging area:
git add .
- Check the status again:

git status

- The modified file should now appear under:
Changes to be committed
- This indicates that the file is staged and ready to be committed.

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git add .
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git status
On branch feature-collaboration
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   1NPDPRO5.JPG
    modified:   README.md
```

Step 12: Commit the Changes

- Create a commit with a meaningful message:
git commit -m "Add collaboration feature"
- The commit stores the changes in the local Git repository.

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git commit -m "Add collaboration feature"
[feature-collaboration e651679] Add collaboration feature
 2 files changed, 10 insertions(+), 1 deletion(-)
 create mode 100644 1NPDPRO5.JPG
```

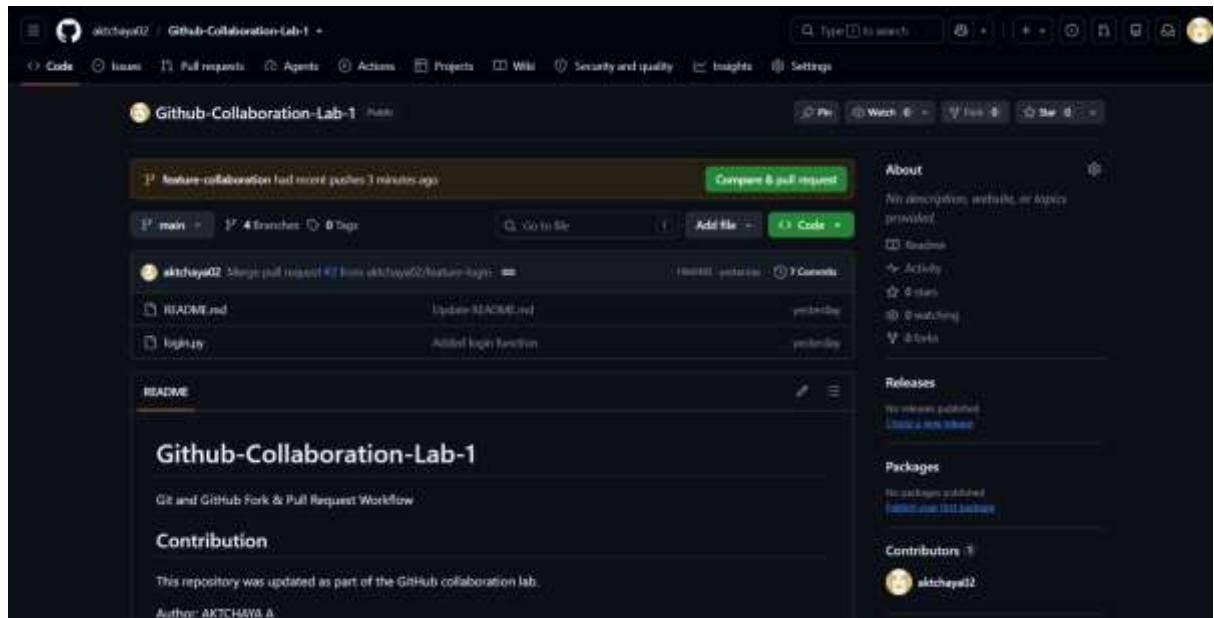
Step 13: Push the Feature Branch

- Push the feature branch to the user's fork on GitHub:
git push origin feature-update
- If your actual branch is named feature-collaboration, use:
git push origin feature-collaboration
- After a successful push, the feature branch becomes available in the forked GitHub repository.

```
PS C:\AKTCHAYA A\Github-Collaboration-Lab-1> git push origin feature-collaboration
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 238.41 KiB | 10.84 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-collaboration' on GitHub by visiting:
remote:   https://github.com/aktchaya02/Github-Collaboration-Lab-1/pull/new/feature-collaboration
remote:
To https://github.com/aktchaya02/Github-Collaboration-Lab-1.git
 * [new branch]      feature-collaboration -> feature-collaboration
```

Step 14: Open the Feature Branch on GitHub

- Open the forked repository on GitHub.
- Select the newly pushed feature branch.
- The modified README.md and the new commit should be visible.
- Click **Compare & pull request**



Step 15: Create the Pull Request

- On the Pull Request page, verify the repositories carefully.
- The Pull Request should be from:

base: main

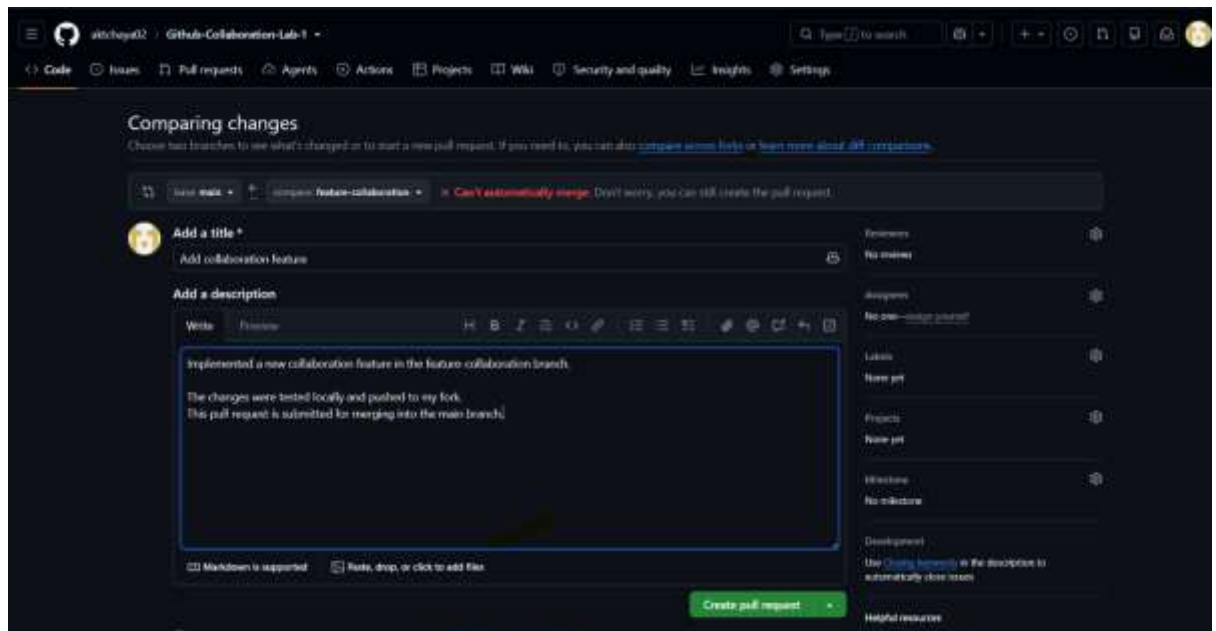
head repository: aktchaya02/Github-Collaboration-Lab

compare: feature-update

- Enter a suitable Pull Request title:

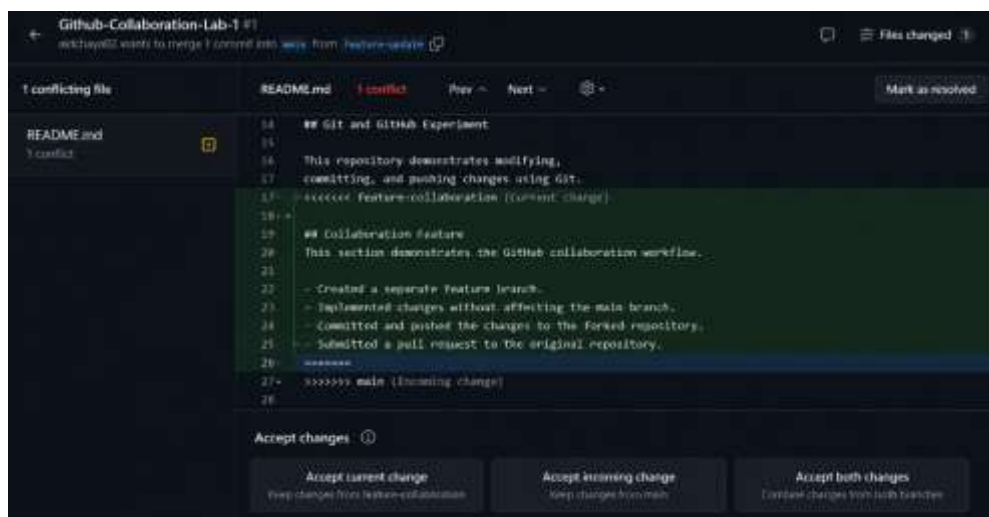
Add collaboration feature

- Add a description explaining the changes.
- Then click: **Create pull request**



Step 16: Pull Request Successfully Created

- After creating the Pull Request, GitHub displays the Pull Request page.
- The page shows that the feature branch from the user's fork is requesting to merge into the main branch of the original repository.
- This confirms that the Pull Request has been successfully submitted.
- Merge conflict will be detected in README.md



Step 17: Resolve the Conflict

- Remove all the important conflict markers from README.md
- Conflict markers like:

<<<<<<<<

=====

>>>>>>>>

- Click:

Mark as resolved

